

Building a Simple Single-Cycle RISC Datapath

AArch64 datapath components, control signals,
and a worked `ldr` instruction path

Dr. J. Burns

Topic 8 Outline

1. Single-Cycle Microarchitecture

- Architectural state and execution within one clock period.

2. State Elements and Core Datapath Blocks

- PC, memories, register file, ALU, and immediate generator.

3. Worked Load Path

- The complete path for `ldr x7, [x5, #8]`.

4. Extending the Datapath

- PC-relative branches and the complete extended datapath.

5. Control Signals and Timing

- Control settings and the single-cycle clock-period limit.

6. Summary and Review

- Summary and self-test questions.

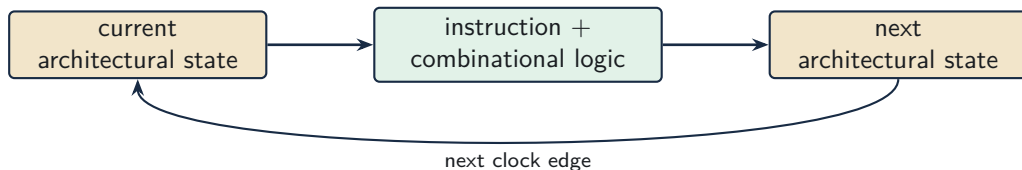
PART 1

Single-Cycle Microarchitecture

Microarchitecture

- The **ISA** defines the instructions visible to software.
- The **microarchitecture** is the hardware organization that executes those instructions.
- This single-cycle datapath implements a small subset of AArch64 instructions.
- The datapath moves values through memories, registers, ALUs, and multiplexers.
- The control unit interprets the instruction and selects the datapath actions.
- Later implementations can use pipelining while preserving the same ISA behavior.

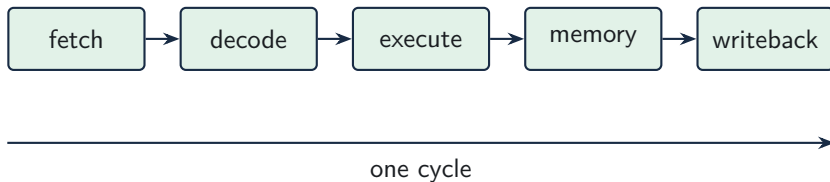
Architectural State as a Finite-State Machine



- State elements hold the visible machine state between cycles.
- Combinational logic computes the next state from the current state and instruction.
- A clock edge commits the new state.

Single-Cycle Execution

- One instruction completes in one clock cycle.
- Fetch, decode, execute, memory access, and writeback all fit in that cycle.
- State is updated at the clock edge at the end of the cycle.
- The clock period must be long enough for the slowest supported instruction.



AArch64 Architectural State

- **PC**: address of the current instruction.
- **Register fields**: 5-bit fields encode register numbers 0–31. Ordinary integer registers are x0–x30.
- **NZCV**: condition flags used by conditional branches. Flag storage and condition-check logic are not shown in the diagrams.
- The PC is modeled as a separate register, not as x15 or r15.
- Encoded register number 31 selects sp or xzr, depending on the instruction form.
- This simplified datapath treats the register file as two read ports and one write port.

The AArch64 PC is separate from its general-purpose register file.

Instruction Subset

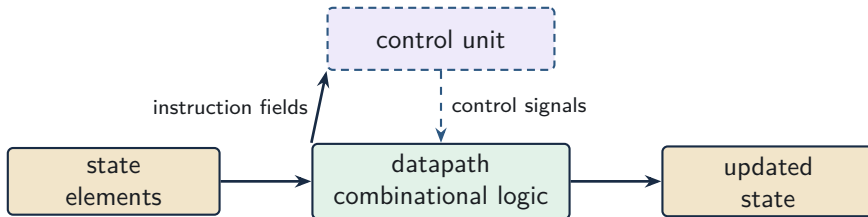
- **Memory:** `ldr xT, [xN, #imm]` and `str xT, [xN, #imm]` using unsigned scaled offsets.
- **Data processing:** unshifted register forms of `add`, `sub`, `and`, and `orr`, plus unshifted immediate `add/sub`.
- **Control flow:** unconditional `b label`.
- The load instruction is traced first because it exercises fetch, register read, immediate generation, ALU address calculation, data memory, and writeback.

Examples use `x0–x30`. Handling `sp/xzr` requires register-31 logic; conditional branches require flag storage and condition-check logic. These additions are not shown.

PART 2

State Elements and Core Datapath Blocks

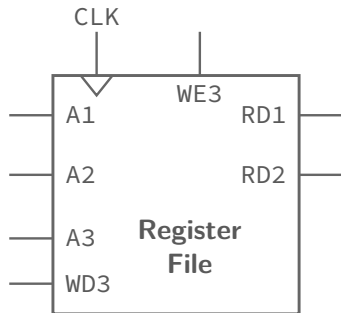
Datapath and Control



- The datapath contains the values and arithmetic.
- The control unit decodes the instruction and:
 - Enables writes to the register file or data memory when required.
 - Selects register or immediate operands and the ALU operation.
 - Selects the immediate field, extension, and scaling.
 - Chooses writeback data and the next PC: PC+4 or a branch target.

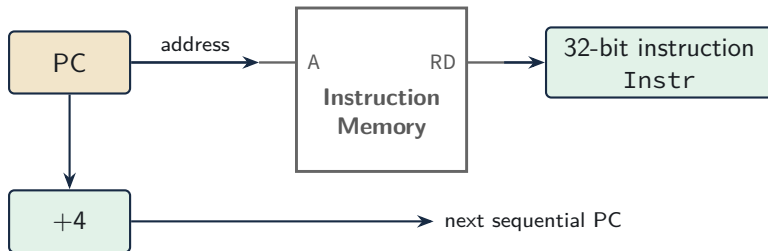
Register File

- A1 and A2: 5-bit register numbers selecting the two reads.
- RD1 and RD2: the two 64-bit register values.
- A3: 5-bit destination register number.
- WD3: the 64-bit value to write.
- WE3: write enable; the selected register changes at the clock edge.



Two read ports and one write port; the read paths are combinational.

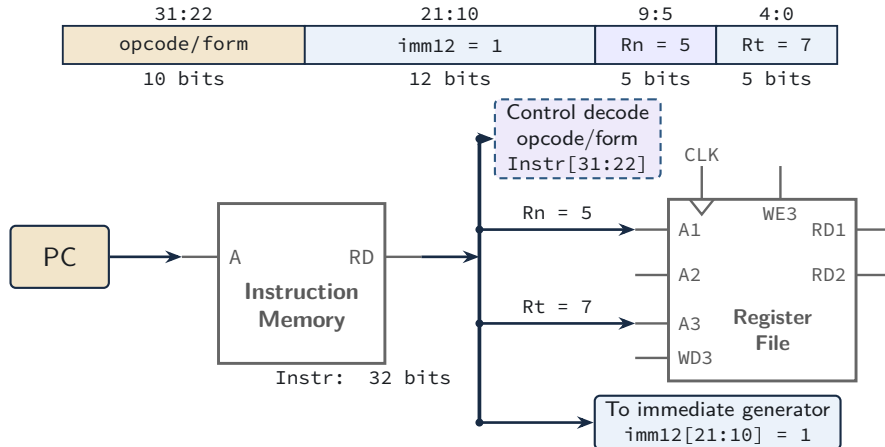
Program Counter and Instruction Memory



- A receives the PC; RD returns the instruction word.
- Instruction memory is read combinatorially in this single-cycle datapath.
- AArch64 instructions are 4 bytes, so the sequential PC value is $PC + 4$.

Parsing the Instruction Word

Example: `ldr x7, [x5, #8]`.



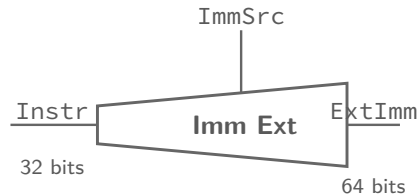
The bar is wiring, with no stored state. For this load: $Rn \rightarrow A1$, $Rt \rightarrow A3$; $A2$ is unused.

Bits [4:0] are Rt for `ldr/str`, and Rd for arithmetic instructions.

Immediate Generator

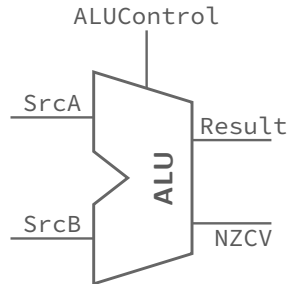
- Instr supplies the instruction bits.
- ImmSrc selects the field and the required extension/scaling mode.
- The selected value is zero-extended or sign-extended to 64 bits.
- Load/store and branch offsets are also scaled when required.
- ExtImm is the resulting 64-bit datapath value.

This is combinational logic: it has no clock input and stores no state.



Arithmetic Logic Unit (ALU)

- SrcA and SrcB supply two 64-bit operands.
- ALUControl selects add, subtract, or a logical operation.
- Result is the 64-bit result or effective address.
- The ALU can also produce NZCV; flag-setting instructions enable a separate flag-register write.



The ALU is combinational. Its operation does not itself update a register.

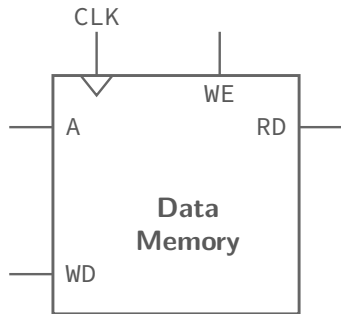
AArch64 Immediate Modes in This Datapath

Instruction form	Immediate field	Datapath value
ldr/str xT, [xN, #imm]	imm12	zero-extended and scaled by access size
add/sub xD, xN, #imm	imm12	zero-extended immediate
b label	imm26	sign-extended, shifted left by 2

ImmSrc selects the field, extension, and scaling. The add/sub row uses unshifted immediates.

Data Memory

- A: the byte address computed by the ALU.
- RD: the 64-bit value read for `ldr xT`.
- WD: the 64-bit register value supplied by `str xT`.
- WE = MemWrite: enables a memory write at the clock edge.
- Reads are combinational in this single-cycle datapath; instruction and data memory are separate.



State Elements



- The PC supplies the address for instruction fetch.
- The register file stores integer register values.
- Data memory is read by `ldr` and written by `str`.
- Writes to architectural state happen at the end of the cycle.

Gold: storage/state

Green: combinational logic

Later diagrams use these coloured blocks and show the connections needed for each instruction.

PART 3

Worked Load Path

Load Example

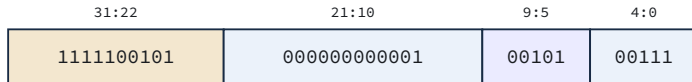
```
ldr x7, [x5, #8]
```

- Read the base address from x5.
- Add the byte offset 8 to form the effective address.
- Read 8 bytes from data memory.
- Write the loaded 64-bit value into x7.
- Update the PC to the next sequential instruction at the end of the cycle.

For a 64-bit AArch64 load with unsigned offset, the encoded imm12 is $8/8 = 1$.

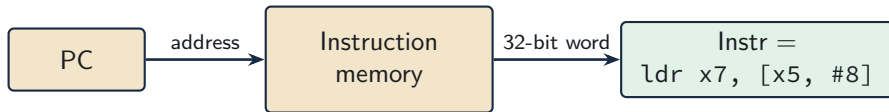
Load Instruction Fields

`ldr x7, [x5, #8]` is encoded as `0xF94004A7`.



Field	Value	Meaning
opcode/form	1111100101	64-bit load, unsigned immediate
imm12	00000000000001	scaled offset 1, meaning 8 bytes
Rn	00101	base register x5
Rt	00111	destination register x7

Step 1: Fetch the Instruction



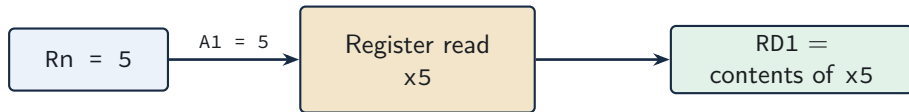
The PC supplies the instruction address. Instruction memory returns the 32-bit AArch64 instruction word.

Step 2: Decode Register Fields

opcode/form	imm12 = 1	Rn = 5	Rt = 7
-------------	-----------	--------	--------

- Rn selects the base register: x5.
- Rt selects the destination register: x7.
- The register file reads x5 while the rest of the datapath computes the offset and address.

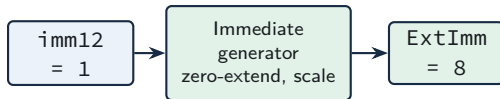
Step 3: Read the Base Register



The instruction names x5; the register file supplies the 64-bit base address stored in it.

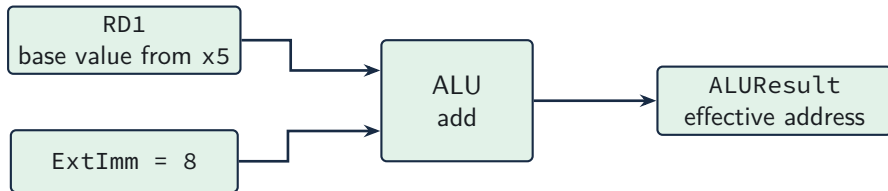
Step 4: Generate the Offset

- Field value: $\text{imm12} = 1$.
- Access size: 8 bytes, because this is `ldr x`.
- Byte offset: $1 \times 8 = 8$.



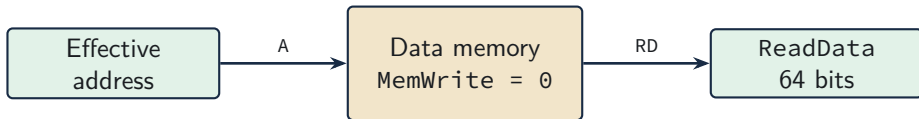
AArch64 unsigned load/store offsets are scaled by the access size.

Step 5: Compute the Effective Address



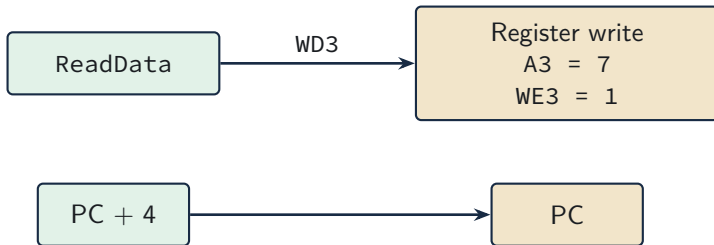
For a load or store, the ALU is used as an address adder.

Step 6: Read Data Memory



The memory read value becomes the candidate writeback value for x7.

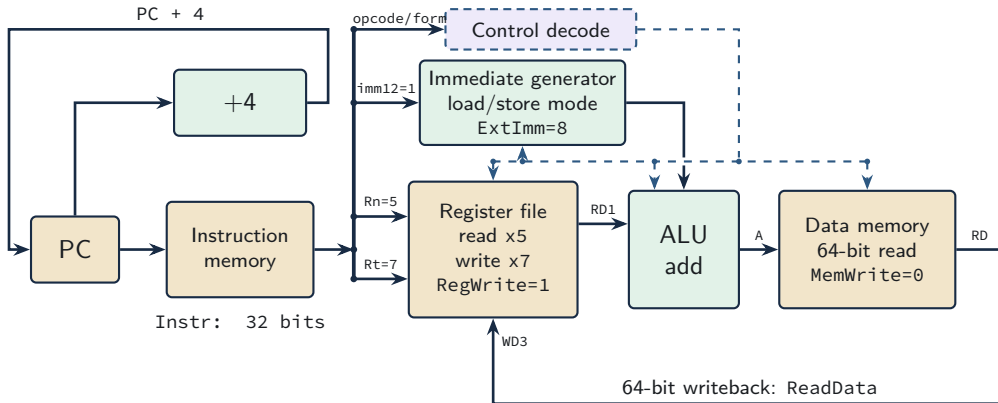
Step 7: Write Back and Update PC



At the clock edge, x7 receives the loaded value and the PC receives $PC + 4$.

Complete Load Datapath

`ldr x7, [x5, #8]`



At the clock edge, x7 receives the 64-bit read value and the PC receives $PC + 4$. Dashed outlines and wires identify control.

Load Path Control Settings

Signal	Setting for <code>ldr x7, [x5, #8]</code>	Effect
RegWrite	1	write loaded value to x7
MemWrite	0	do not write data memory
MemToReg	1	writeback comes from data memory
ALUSrc	1	ALU second input is the extended offset
ALUControl	add	compute $\text{base} + \text{offset}$
ImmSrc	load/store scaled	produce byte offset 8
PCSrc	0	next PC is $PC + 4$

These control values select one path through the same hardware.

PART 4

Extending the Datapath

Branching in AArch64: Forward and Backward

A branch changes which instruction is fetched next. `b label` transfers control unconditionally to `label`.

The branch instruction supplies the base address

$$\text{Target} = PC_{\text{branch}} + \text{signed byte offset}$$

PC_{branch} is the address of the `b` instruction itself.

Forward: skip an instruction

```
    b ahead      // 0x1000
    add x0,x0,#1  // 0x1004
ahead: sub x1,x1,#1 // 0x1008
```

$$0x1008 - 0x1000 = +8 \text{ bytes}$$

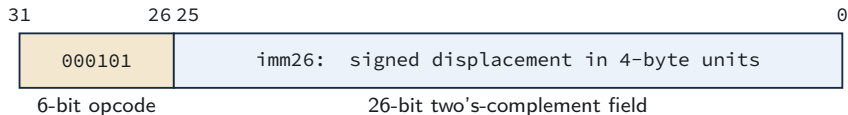
The assembler calculates the displacement from the label. Each A64 instruction occupies 4 bytes.

Backward: repeat instructions

```
again: add x0,x0,#1 // 0x2000
      sub x1,x1,#1  // 0x2004
      b again      // 0x2008
```

$$0x2000 - 0x2008 = -8 \text{ bytes}$$

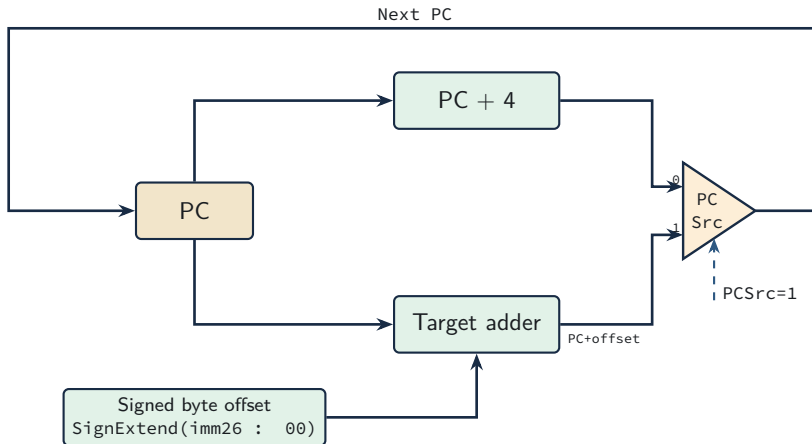
B Instruction: Encoding the Offset



- The label is represented by a displacement, not an absolute address.
- $\text{imm26} = (\text{Target} - \text{PC_branch}) / 4$.
- Append two zero bits, then sign-extend to 64 bits:
 $\text{offset} = \text{SignExtend}(\text{imm26} : 00)$. Here $:$ joins bit fields.

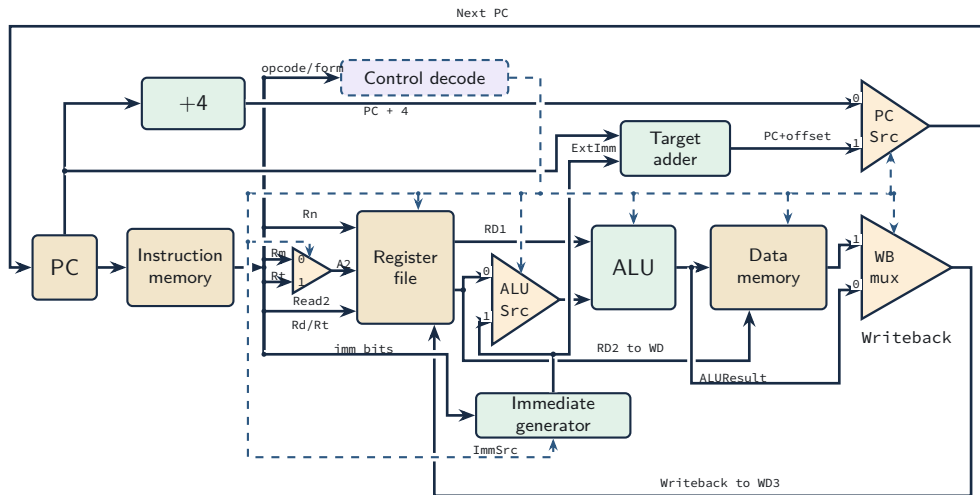
Byte offset	Signed imm26	Encoded field	32-bit instruction
+8	+2	0x00000002	0x14000002
−8	−2	0x3FFFFFFE	0x17FFFFFFE

Branch Path: b label



For b, $PCSrc=1$ selects $PC_{branch} + offset$. At the clock edge, this target replaces the value in the same PC register.

Complete Extended Datapath



Triangles: multiplexers; dashes: control. The same hardware supports add and str; see Self-Test 2.

PART 5

Control Signals and Timing

Control Signals

Signal	Meaning
RegWrite	enable a write to the register file
MemWrite	enable a write to data memory
MemToReg	choose memory data or ALU result for writeback
ALUSrc	choose register data or immediate as ALU input B
ALUControl	select add, subtract, and, or other ALU operation
ImmSrc	select the immediate field, extension, and scaling
Read2Src	choose Rm or Rt for the second register read
PCSrc	choose sequential PC or branch target

Example Control Settings

Instruction	RegW	MemW	MemToReg	ALUSrc	ALU	ImmSrc	Read2Src	PCSrc
ldr xT, [xN, #imm]	1	0	1	1	add	load/store	X	0
str xT, [xN, #imm]	0	1	X	1	add	load/store	1	0
add xD, xN, xM	1	0	0	0	add	X	0	0
sub xD, xN, xM	1	0	0	0	sub	X	0	0
add xD, xN, #imm	1	0	0	1	add	add/sub imm	X	0
b label	0	0	X	X	X	branch	X	1

Read2Src: 0 selects Rm; 1 selects Rt.

X means not used: its setting does not affect this instruction.

Single-Cycle Limitations

- Hardware is easy to reason about, but timing is inefficient.
- Every instruction waits for the worst-case datapath delay.
- Simple instructions cannot finish early.
- Some hardware must be duplicated when two values are needed in the same cycle.
- The next topic reuses the same datapath ideas but overlaps instructions in a pipeline.

Looking Ahead

- The single-cycle datapath shows the full instruction path in one cycle.
- Pipelining splits that path into stages so several instructions can be in progress at once.
- The next topic uses the same fetch, decode, execute, memory, and writeback structure to build the 5-stage RISC pipeline.
- Hazards appear when overlapped instructions need the same data or control decision.

PART 6

Summary and Review

Topic 8 Summary

- A single-cycle datapath computes the next architectural state in one clock period.
- The state elements shown store the PC, integer register values, and memory contents.
- Instruction fields select register ports, immediate-generation modes, and control settings.
- The `ldr x7, [x5, #8]` path uses instruction fetch, register read, immediate scaling, ALU address calculation, memory read, and register writeback.
- The same datapath is extended with multiplexers and control signals for stores, ALU instructions, and branches.
- `b` adds a signed PC-relative byte offset to the address of the branch instruction; `imm26` encodes that offset in 4-byte units.

Self-Test 1: Datapath Basics

- ❶ In a single-cycle datapath, when are state elements updated?
- ❷ Why is the PC modeled separately from the AArch64 general-purpose register file?
- ❸ For `ldr x7, [x5, #8]`, which fields select the base and destination registers?
Which register-file inputs receive them?
- ❹ Why is the encoded `imm12` equal to 1 rather than 8 for that instruction?
- ❺ What is the role of the ALU in a load or store instruction?
- ❻ What does the vertical instruction bar represent? Does it store state?

Self-Test 2: Control and Timing

- 1 Identify the four triangular multiplexers in the complete extended datapath. What two data inputs and one output does each have? Which signal controls the writeback multiplexer?
- 2 Why does `str` assert `MemWrite` but not `RegWrite`?
- 3 Trace `add x1, x2, x3` and `str x7, [x5, #8]` through the complete diagram. For each, identify the register fields, ALU inputs, selector settings, memory action, register write (if any), and next PC.
- 4 Why does a single-cycle implementation have a long clock period?
- 5 Which datapath components are used by `add x1, x2, x3` but not by `b label`?
- 6 A `b` at `0x2008` targets `0x2000`. What are the byte offset, signed `imm26`, and next PC? Which address is the base?